5 — Vibecoding in practice

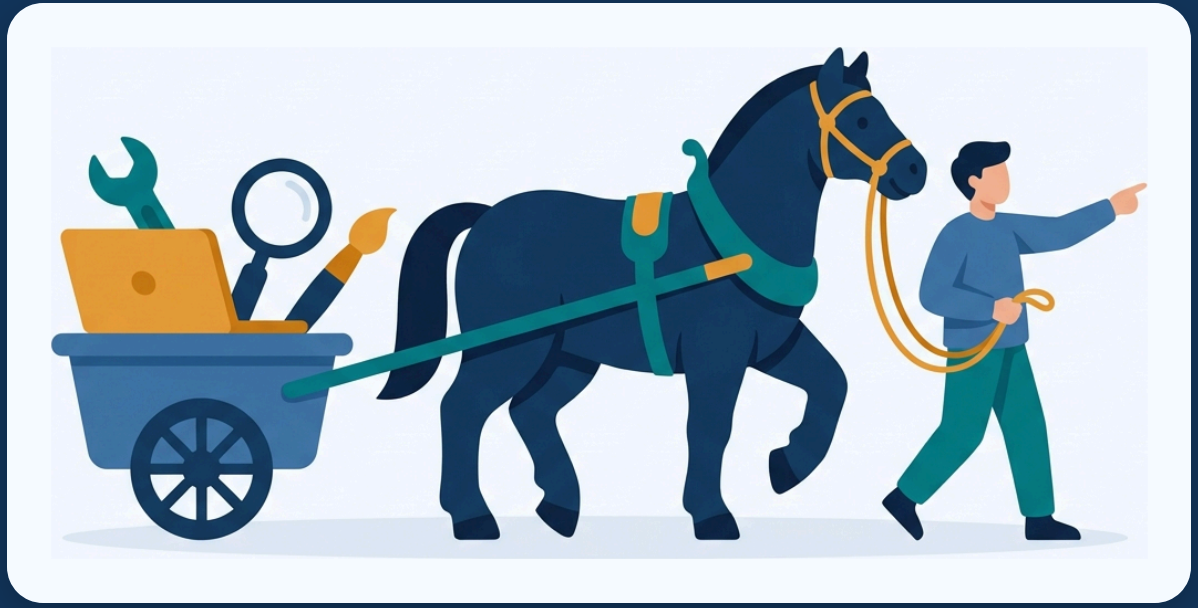
From a goal to a working artifact

Tomáš Vičar
Department of Biomedical Engineering · FEEC, Brno University of Technology

2026

1 · A model with tools

> Give the agent a goal, a working environment and a way to check the result.



What do we mean by vibecoding?

> **Build by describing what you want in ordinary language. Let AI implement it, try the result, and ask for changes.**

You describe

“Make a web page to explore these measurements.”

AI implements

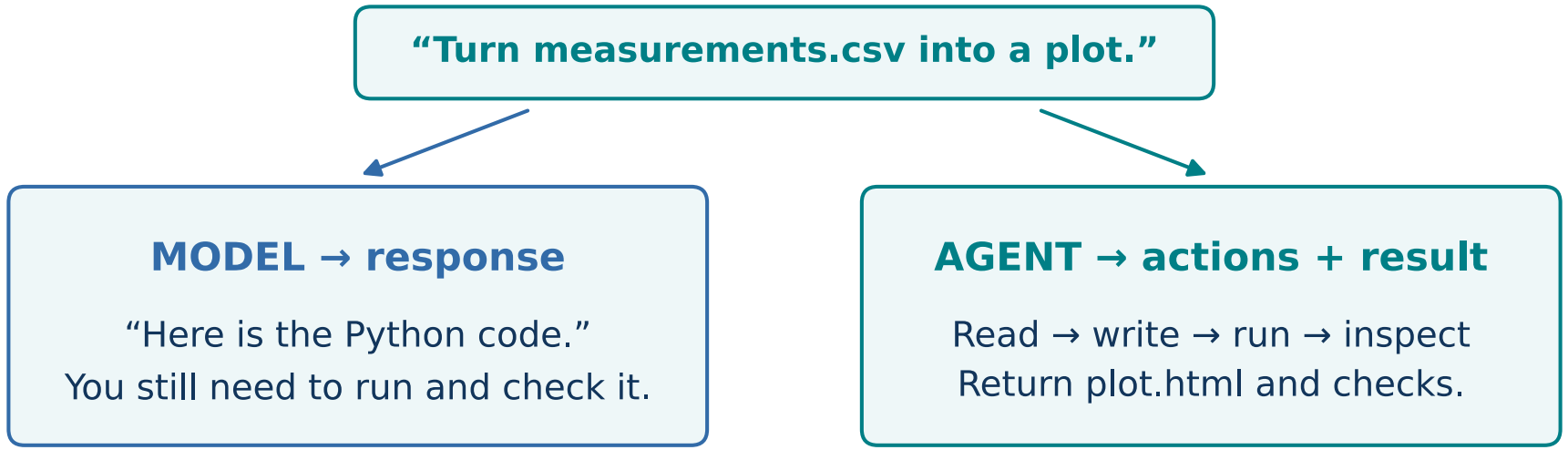
Writes the code, runs it and fixes the errors.

You try and steer

“Add a filter. Check that the counts are right.”

In this course: describe → build → check → improve. You remain responsible for the result.

Model vs. agent

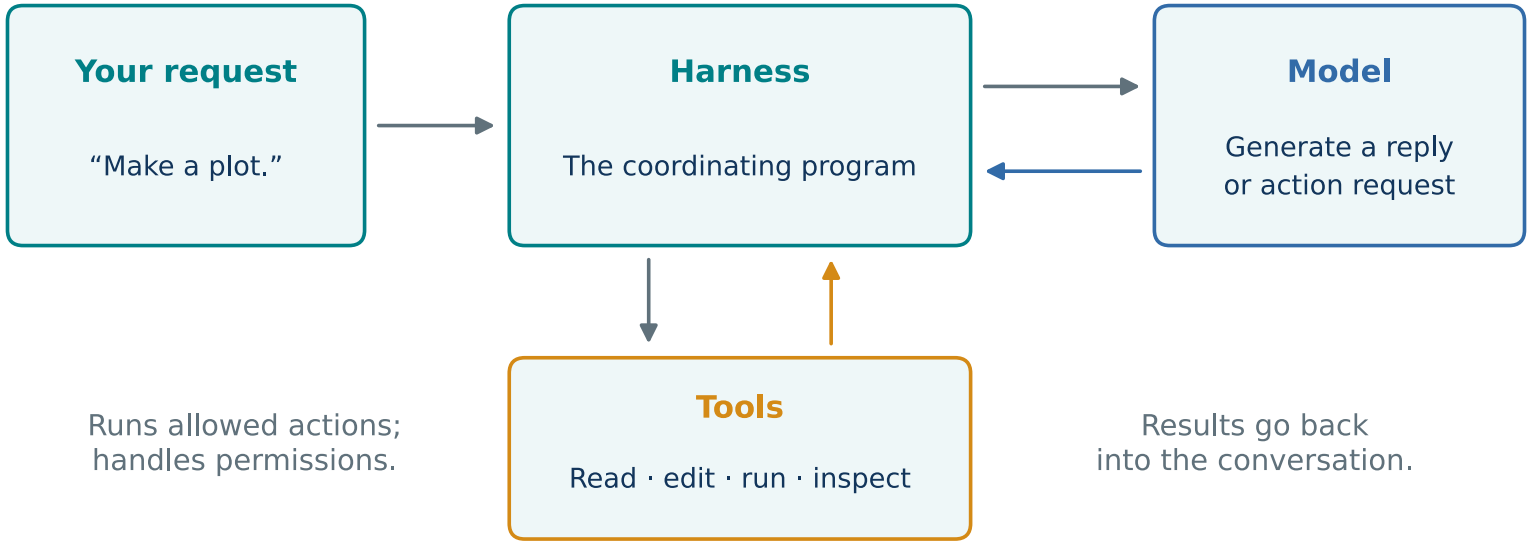


The agent uses a model, plus software that can carry out actions.

- **Model:** gives you the plotting code as text. You run it.
- **Agent:** writes the code, **runs it**, fixes the error and hands you `plot.html`.

> **Model** = the trained network that writes text. **Agent** = a model plus software that can act.

The harness: the program that lets the model act

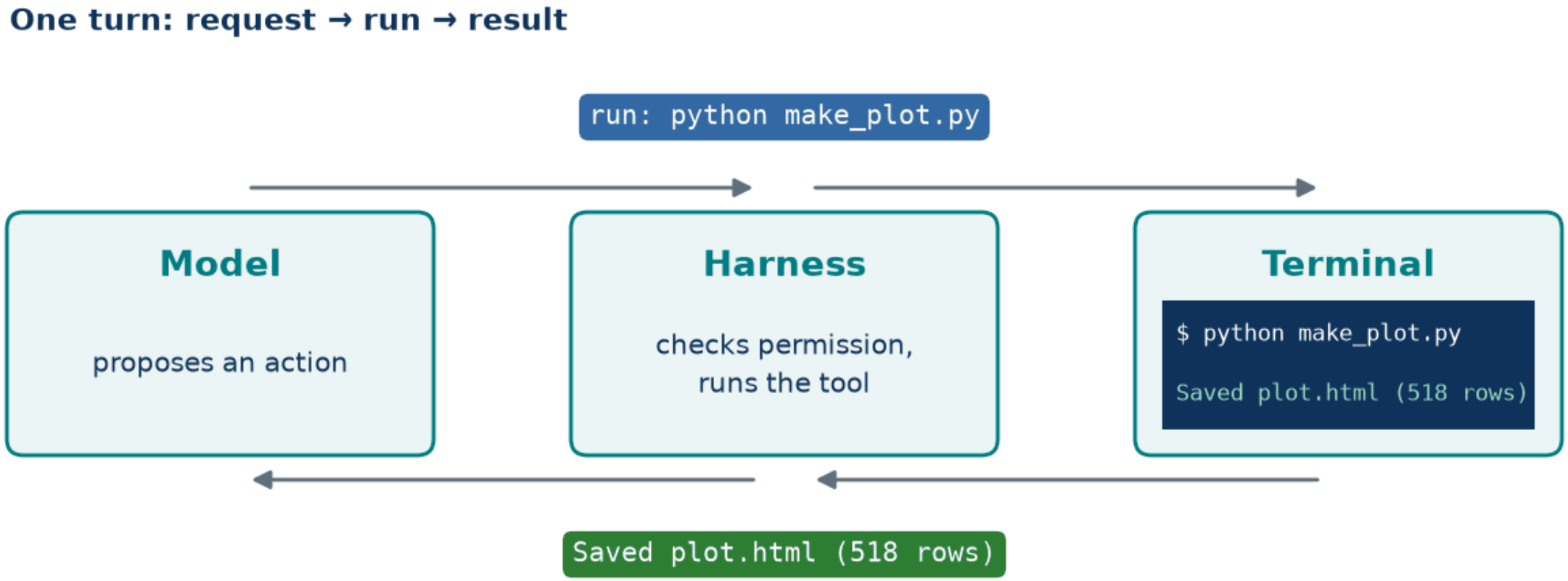


- **Harness** = the program you install (Claude Code, Codex). It sends your request to the model and **carries out** the actions the model asks for.

Agent = model + harness + tools. The model decides what to do; the harness does it.

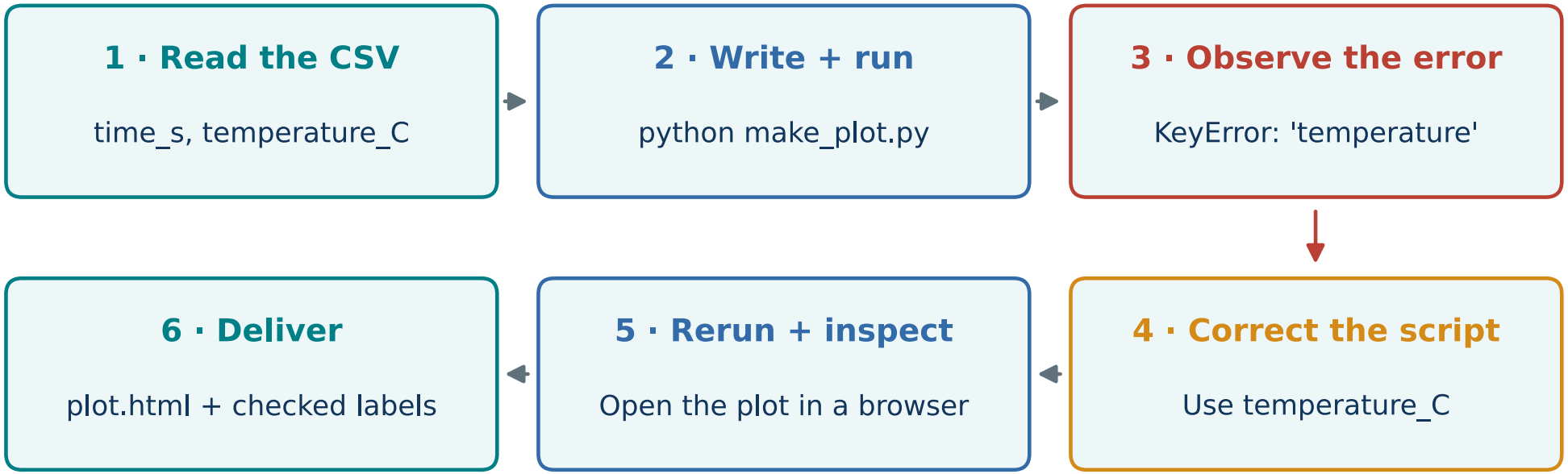
A tool is an action the harness performs for the model

Ordinary tools: read a file · edit a file · run a command (e.g. `python`, `git`) · open a browser.



Terminal = a text window for typing commands · **shell** = the program that runs them · **CLI** (command-line interface) = a program you use by typing.

One task, several turns through the loop



- **One turn:** the model proposes an action → the harness runs it → the result comes back.

> You give the **goal**. The agent chooses steps; **tool results** guide its next attempt.

Context: what the model sees on this turn

Your request

“Plot the measured temperature over time.”

Relevant information

Column names, units, instructions and earlier decisions.

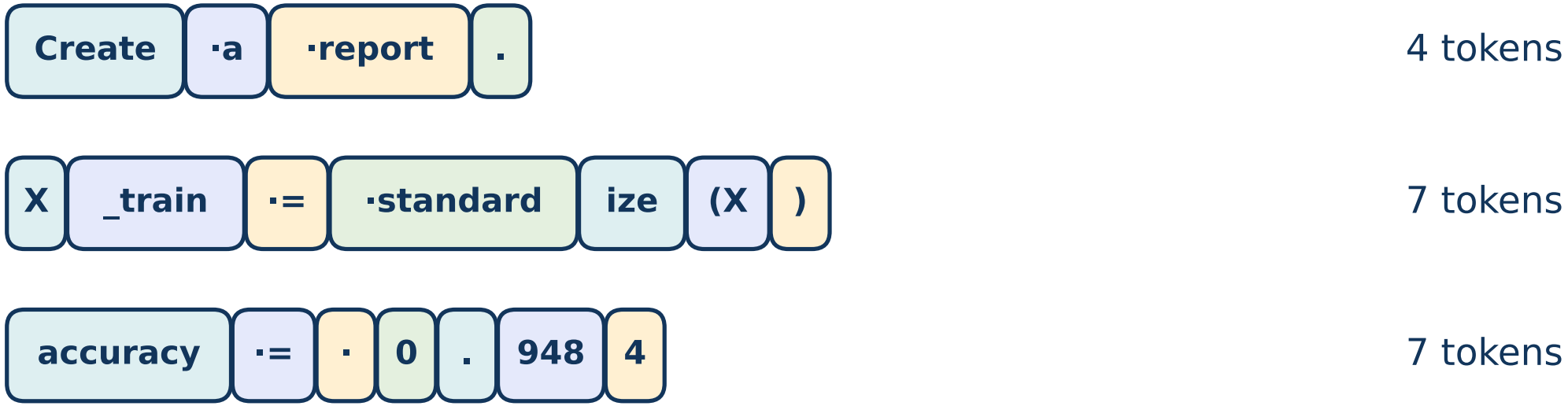
Tool results

A file excerpt, an error message or a screenshot.

Context = everything the model reads before this answer. Like a desk: only what lies on the desk counts, not the files in the cabinet (your disk).



Tokens are pieces of the input and output



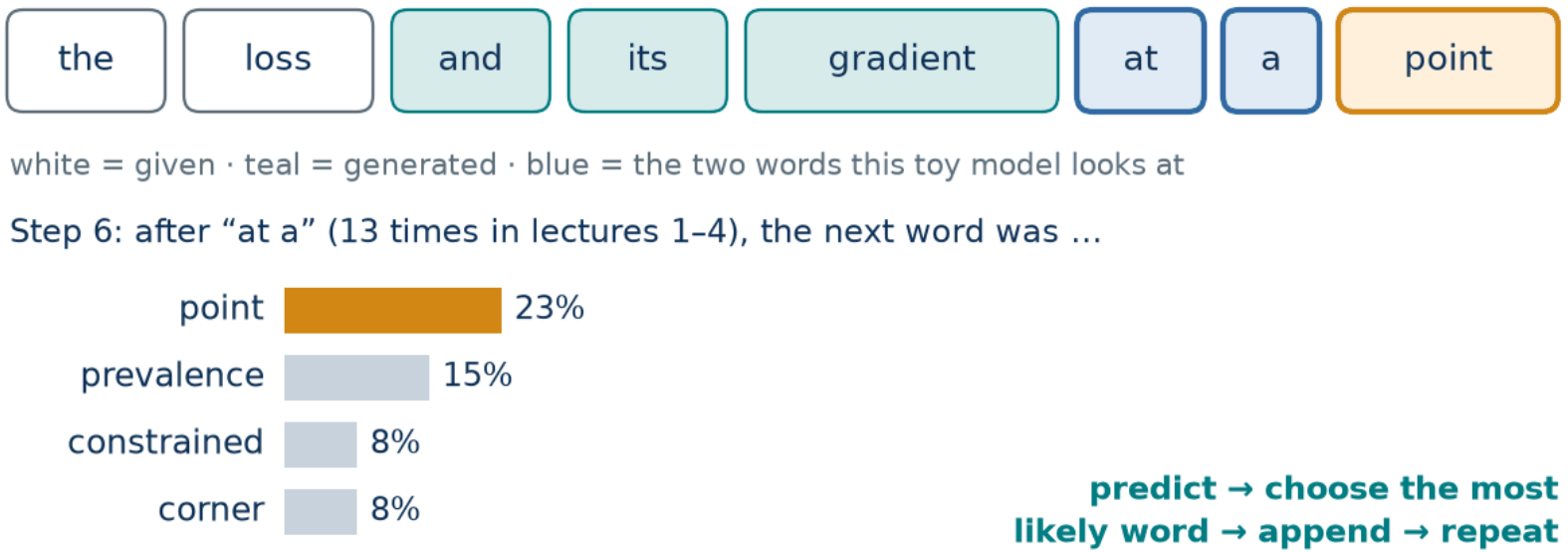
Measured with o200k_base; · marks a space. Tokenizers differ between models.

- **Token:** a word, part of a word, punctuation or another text piece.
- **Context window** = how many tokens the model can read at once. API prices are counted in tokens too.

The model predicts the next token

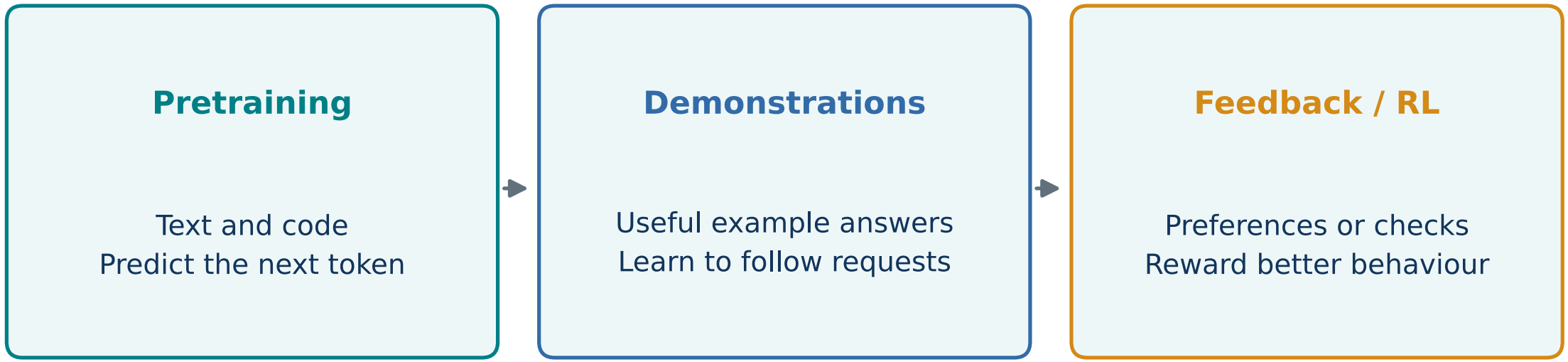
$$p_\theta(t_{k+1} \mid t_1, \ldots, t_k)$$

In words: given all tokens so far, how likely is each possible next token? **Generation:** choose one, append it, repeat.



A **transformer neural network** usually makes these predictions. Its architecture comes in the **eleventh lecture**.

How the model learned: three training stages



A simplified training recipe; a conversation supplies context, not new weights.

Training changed the weights once. A correction in today's chat is only **context**: it is not learned for tomorrow — **write it into a file**.

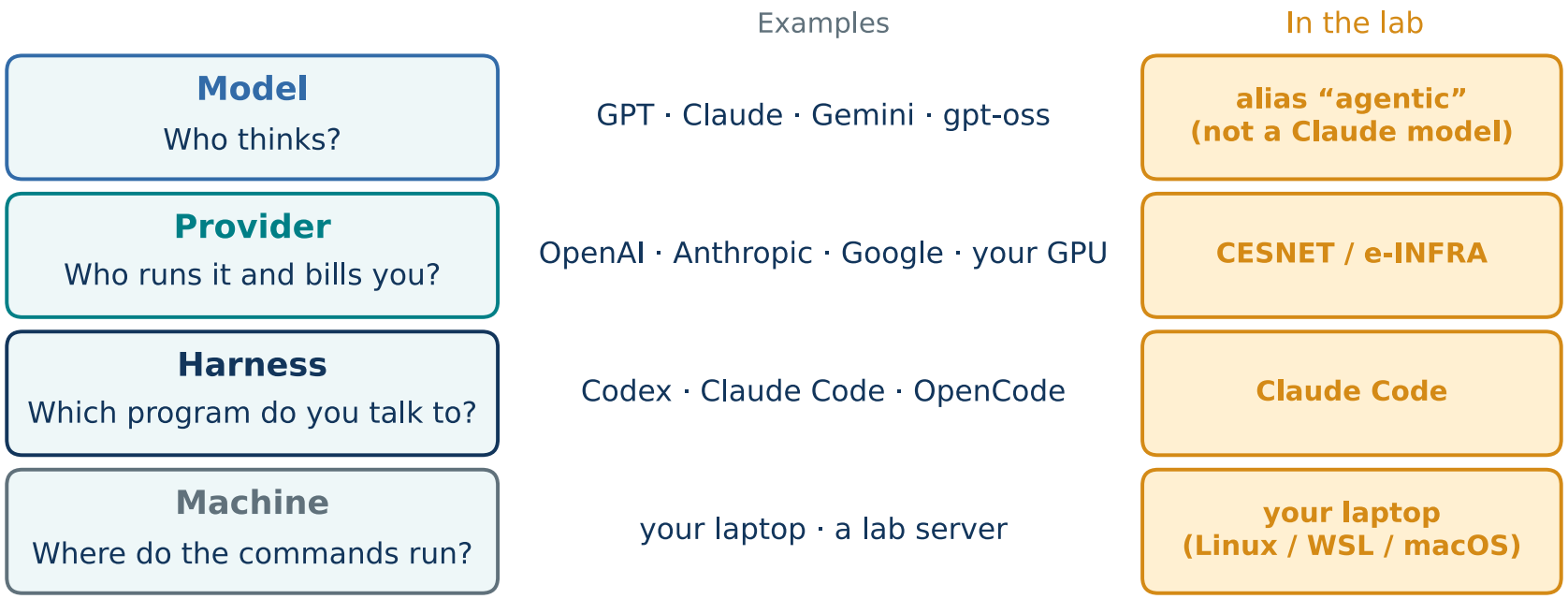
Ouyang et al.: instruction tuning and RLHF

2 · Choose a place to work

> Four separate choices: which model, who runs it, which program you talk to, and which computer runs the commands.



Four layers: model, provider, harness, machine



- **Endpoint** = the web address of a model service (CESNET gives you one).
- **API key** = a secret password that lets your harness use that service.

Providers and harnesses: a practical map

Harness	Why try it	Ask before choosing
Codex · OpenAI	Terminal, code editor and desktop app	ChatGPT plan or API key? Which model?
Claude Code · Anthropic	Terminal agent (used in the lab)	Claude plan, API key or another provider?
Antigravity CLI · Google	Google's terminal agent for individuals	Enterprise and API-key users keep Gemini CLI
Copilot CLI · GitHub	Works alongside your GitHub projects	Which Copilot plan and models?
OpenCode · many providers	Connect almost any model, even a local one	Does that model use tools reliably?

A map, not a ranking. Google: Gemini CLI → Antigravity CLI. Check access and model compatibility.

More harnesses: OpenClaw and Hermes

OpenClaw — a personal assistant you host

- Runs **all the time** on a computer you control.
- You message it from a terminal, a web page or a chat app such as Telegram; it acts on that computer.
- Useful for **one assistant reachable from any device**.

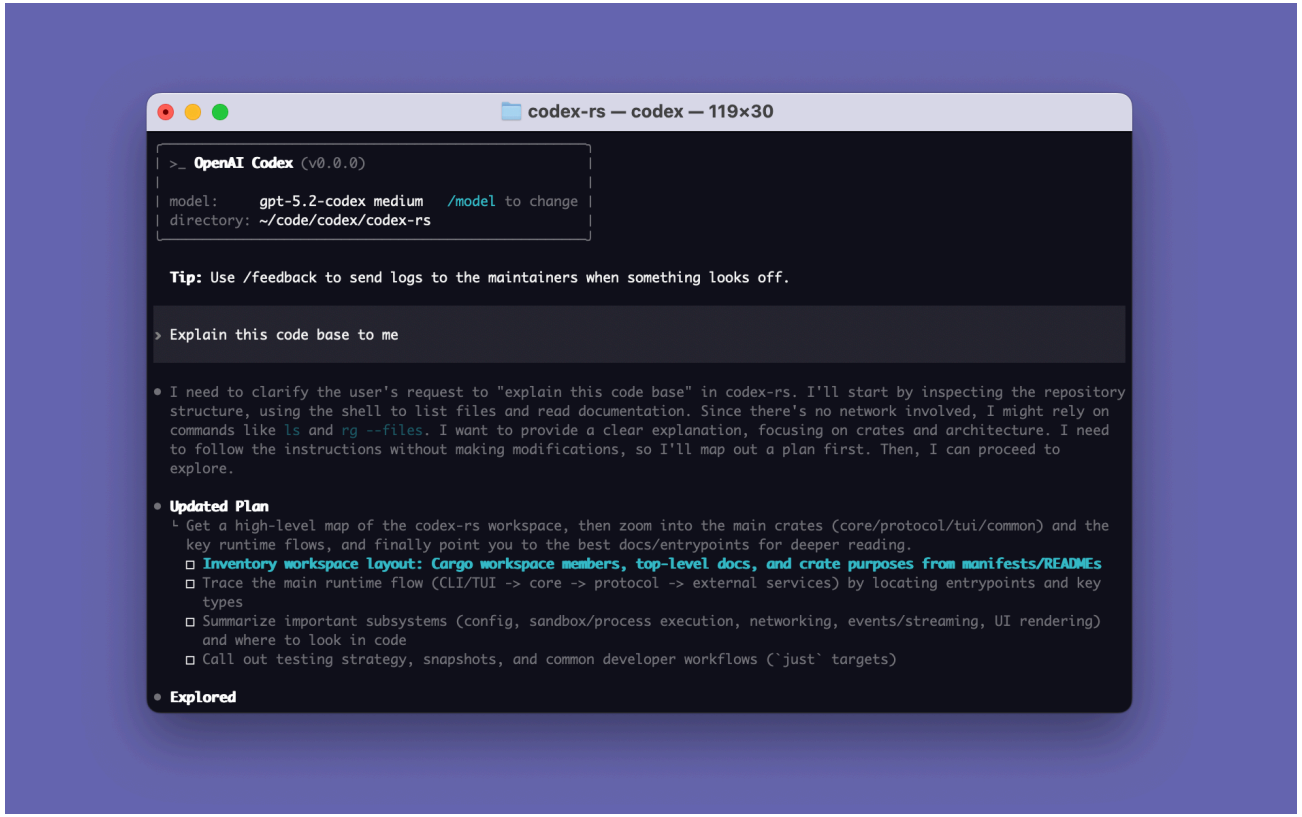
Hermes Agent — a terminal agent that remembers

- **Remembers** earlier sessions and saved procedures.
- Can repeat a task **on a schedule**, e.g. every morning.
- Useful for **recurring tasks that continue across days**.

Both are **harnesses, not language models**. Choose a supported model provider and the tools they can access.

OpenClaw · Hermes Agent. Current capabilities checked 30 Sep 2026.

Terminal: a conversation beside the commands



Recommended for this course

Start in your project folder:

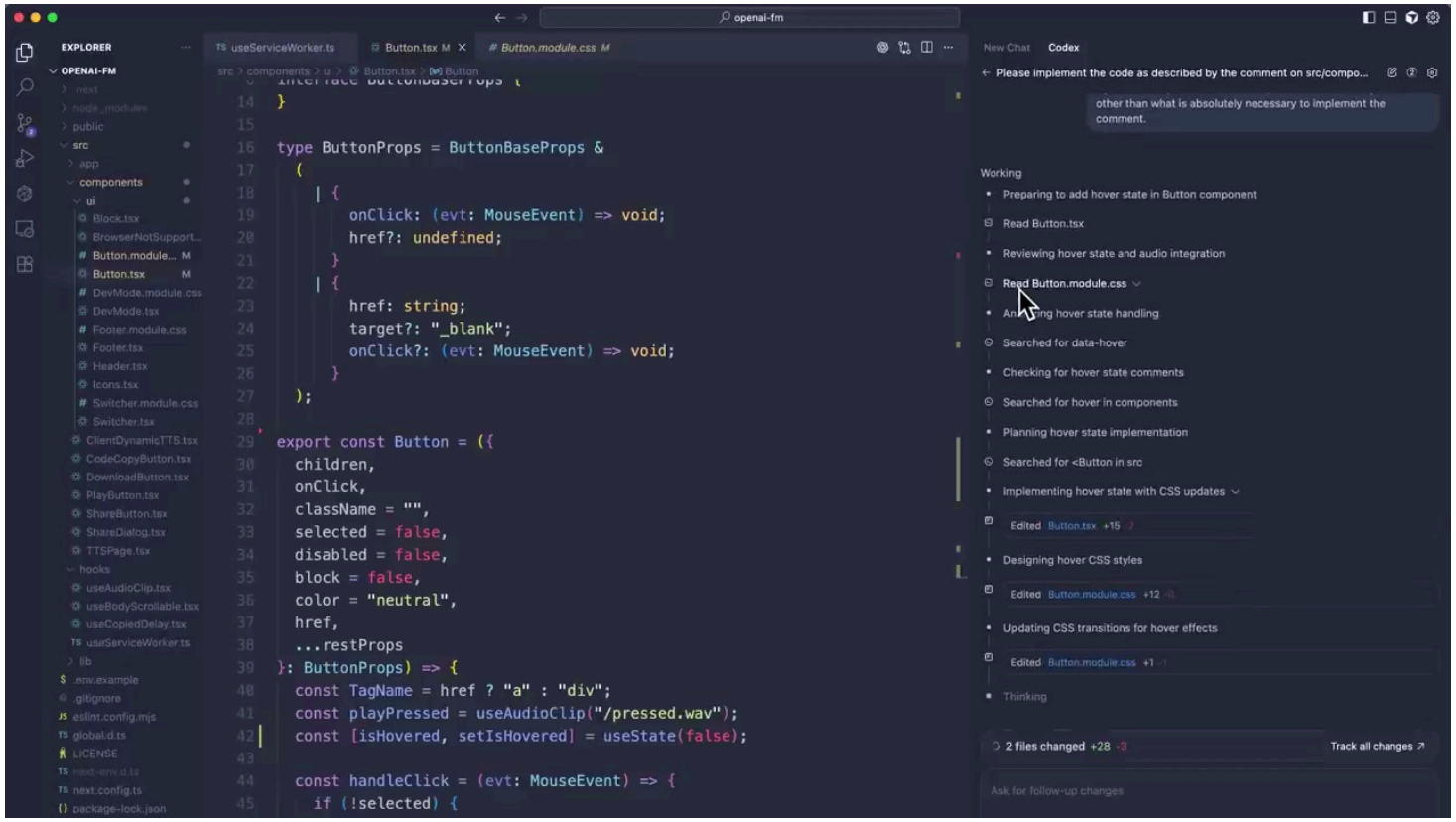
```
cd my-project
codex
# or: claude
```

`cd` = go into the folder.

Also works over **SSH** (log in to another computer's terminal; later today).

OpenAI: Codex CLI screenshot · illustrative interface; displayed model/version is not a recommendation.

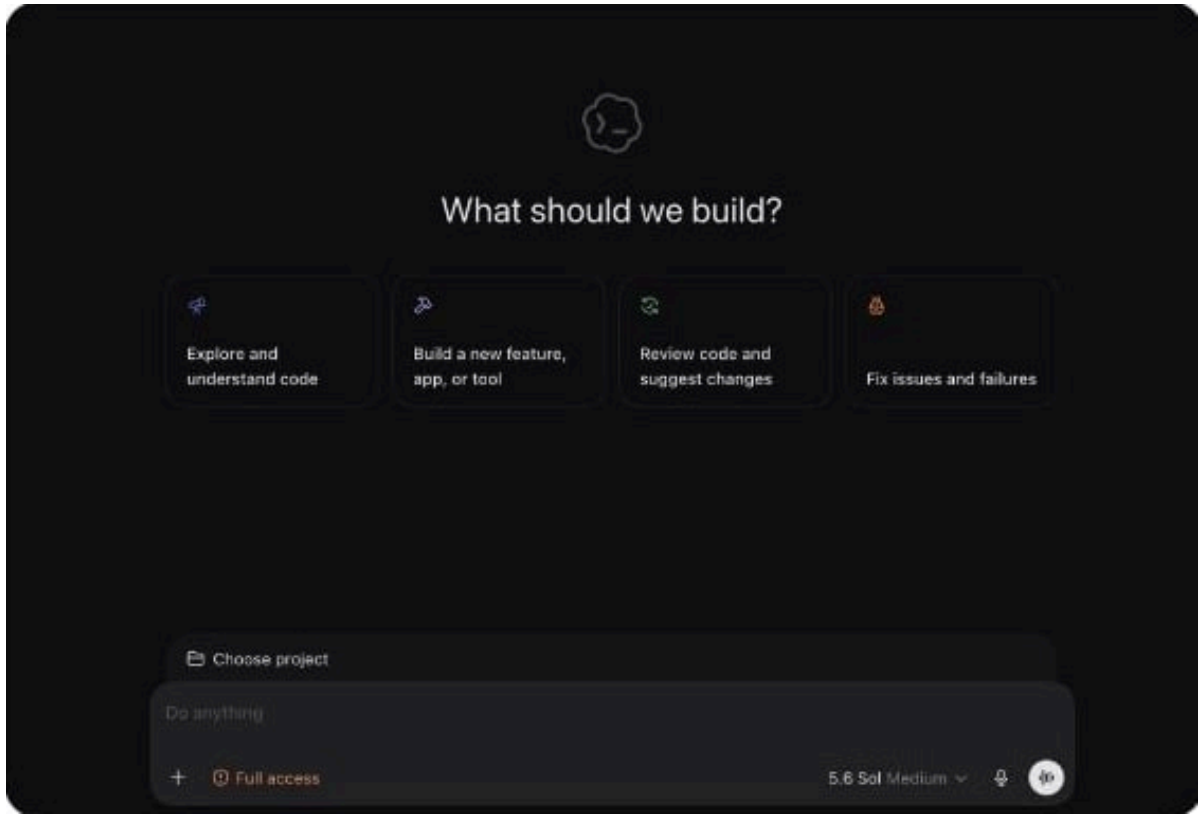
Editor: files, code and the agent in one view



- **VS Code extension:** the agent inside your code editor — read the code, see its changes and steer it in one window.

OpenAI Developers: Codex IDE demonstration · frame at 02:00.

Application: organise tasks and review results



A dedicated graphical workspace

Choose a project → start a task → review the changes.

Keep **several tasks side by side**, come back to them later and **review changes visually**.

OpenAI: Codex desktop example · product example, not a feature-parity claim.

Permission modes: let the agent work

Example: the agent wants to run `pip install pandas`. Who says yes?

Mode / workflow	What happens	When to use it
Plan / read-only	It only reads and proposes; nothing is changed	An unclear task or first look
Ask / default	It asks, **you** press yes or no	Close supervision; the lab with CESNET
Auto — recommended	Automatic checks decide eligible permission requests	Everyday work in a defined project
Full access / bypass	It just runs, **with no check**	Only on a separate, isolated machine

Course recommendation: Auto mode, with a clear scope, recoverable files and checked results.

Auto mode: review actions without constant interruptions



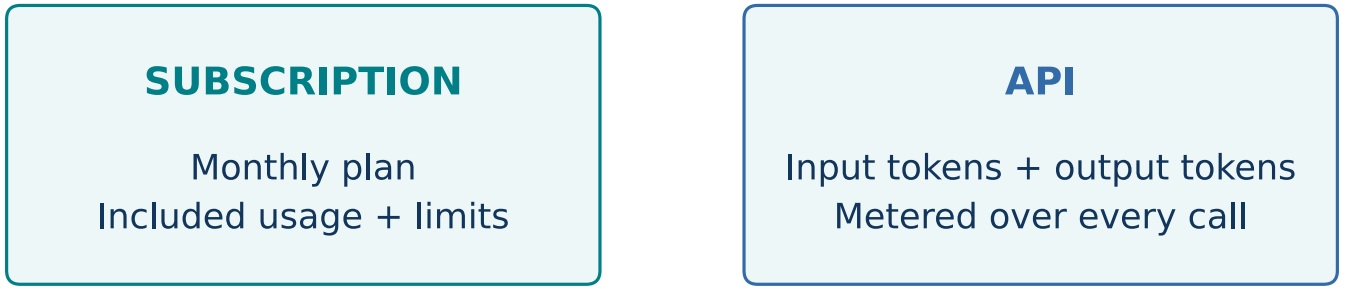
Codex example: automatic review keeps the sandbox boundary.

```
codex --approve-for-me
claude --permission-mode auto
```

In the lab (CESNET models) Auto is not available: start with `claude --permission-mode acceptEdits` (edits go through, commands still ask) or `default`.

Choose your agent, not both commands. Claude Auto needs a supported Claude model; the CESNET aliases are not Claude models. Codex Auto-review · Claude modes

Subscription or API: which meter is running?



Same terminal application, different login route and billing meter.

- **ChatGPT:** Plus **$20/month**; Pro tiers **$100 / $200 / $500**.
- **Claude:** Pro **$20/month**; Max **$100 / $200**. Higher tiers raise limits.
- **API key:** a secret password that lets a program use the model; **you pay per token**, separately. A chat plan is **not API credit**.

Heavy agent use: API bills can be much higher than a subscription. Compare the plan's included usage and limits.

USD examples checked 30 Sep 2026; region, tax and billing interval vary. OpenAI plans · Claude plans

API cost: count the whole agent loop

Example model	Input / 1M tokens	Output / 1M tokens	Example run*
GPT-6 Astra · OpenAI	$10	$50	**$2.00**
Claude Fable 5.1 · Anthropic	$10	$50	**$2.00**
Claude Opus 5.5 · Anthropic	$4	$20	**$0.80**

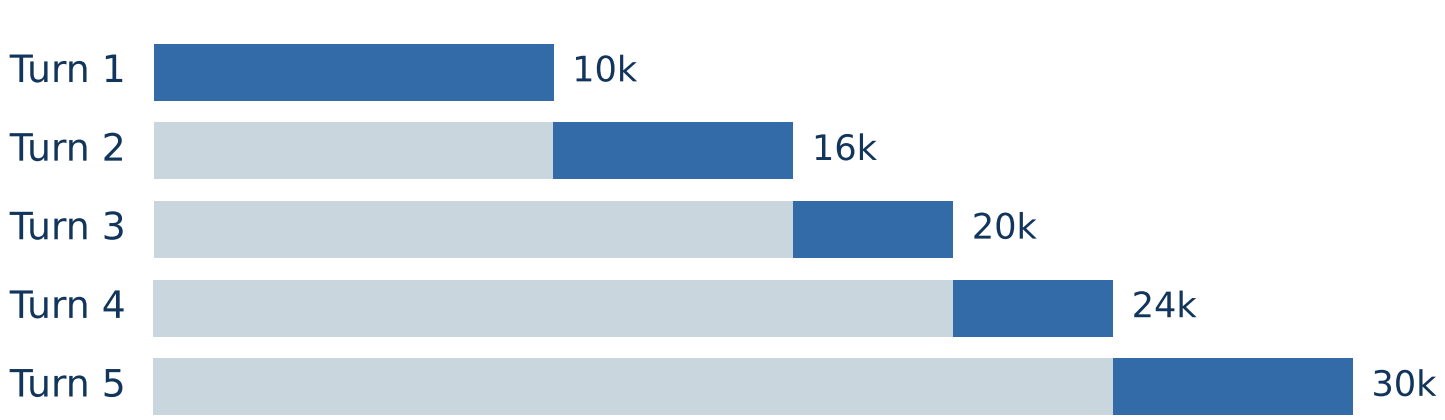
***100,000 input + 20,000 output tokens**, summed across all calls; standard prices, no caching discounts.

Why so much input? Every turn resends the conversation so far.

> Opus 5.5 example: **0.1 × $4 + 0.02 × $20 = $0.80**. One hundred such runs: **$80**.

Current examples, checked 30 Sep 2026: GPT-6 Astra · Fable 5.1 · Opus 5.5. Standard API rates; no quality ranking.

Every turn resends the whole conversation



- **Every turn, the model reads the whole conversation again**, and the input tokens of all of it are billed.
- **So the input adds up:** five turns of 10k–30k tokens are the **100,000 input tokens** of the example.

Short sessions and small tool outputs are cheaper. **Caching** makes the repeated part cheaper, not free.

Why run a model locally?

Control

Choose weights, runtime and where data is processed

Independence

Work offline after setup; no hosted token meter

Responsibility

Pay for hardware and power; maintain and evaluate it

Local is a deployment choice, not a quality guarantee.

- **Open weights:** model files you can download and run yourself, e.g. with Ollama; check the licence.
- **Agent on your laptop ≠ model on your laptop.** Claude Code on your laptop still sends prompts and file excerpts to a cloud model.

How good are local models for agent work?

Often useful

Explain code, transform text, draft tests, make a bounded edit.

Harder to get right

Long tool chains, unfamiliar libraries, recovery from repeated errors.

Try the same small task

1. Fix a bug with a known failing test.
2. Run and inspect the result.
3. Compare success, time and retries.

Use the **same harness and checks** where compatible.

Judge **model + quantisation (fewer bits per weight) + harness + task**. Fitting in memory ≠ useful quality.

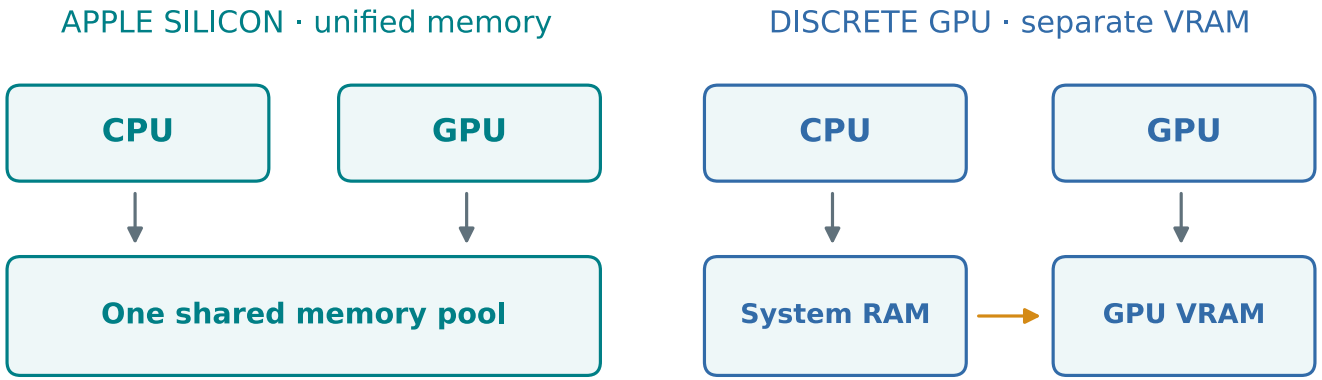
Local hardware: weights must fit before speed matters

Scale / example	Memory guidance	Practical interpretation
gpt-oss-20b	≥16 GB VRAM / unified memory	An accessible local starting point
gpt-oss-120b	≥60 GB starting point; **80 GB GPU** reference	Large-memory workstation or server
70B at 4 bits	35 GB for packed weights **alone**	Add runtime, cache and working memory
DeepSeek V3.2 · 671B, 4 bits	335.5 GB for packed weights **alone**	Large-memory / multi-accelerator system

70 B (billion) **parameters × 4 bits ÷ 8 = 35 GB** just to store them. **VRAM** = the GPU's own memory.

OpenAI local runtime guide · DeepSeek model card. Last two rows: weight arithmetic, **not tested configurations**. Decimal GB.

Apple unified memory: capacity versus speed



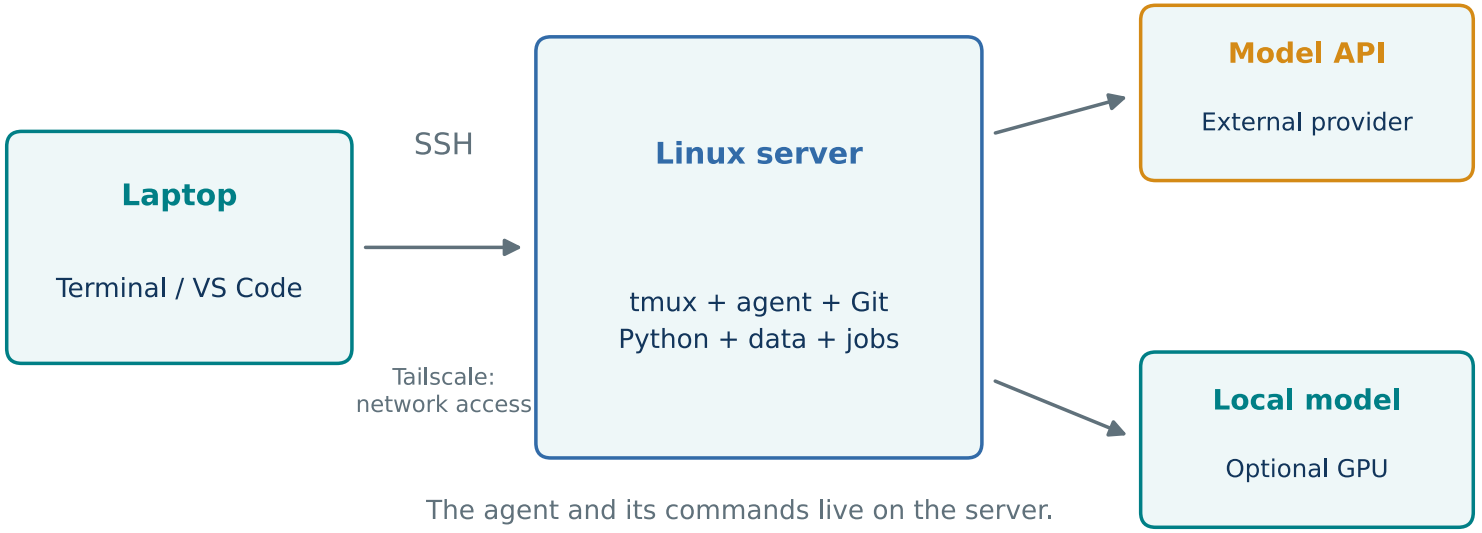
Fit depends on usable capacity; speed also depends on bandwidth and compute.

Hardware example	Memory capacity	Memory bandwidth
Mac Studio **M3 Ultra** (launch configuration)	Up to **512 GB shared**	**819 GB/s**
NVIDIA **RTX 5090**	**32 GB VRAM**	**1,792 GB/s**

Capacity decides if it fits; **bandwidth** (memory read speed) is one factor in speed.

Apple launch / bandwidth · NVIDIA hardware figures. Hardware examples, not current stock or inference benchmarks.

Long jobs: let a server keep working



- **Server:** a computer that stays on; keeps the data, tools and long jobs.
- **Laptop:** connect, give a task, check progress — then disconnect.
- **Linux / WSL** (a Linux terminal inside Windows): the same commands everywhere.

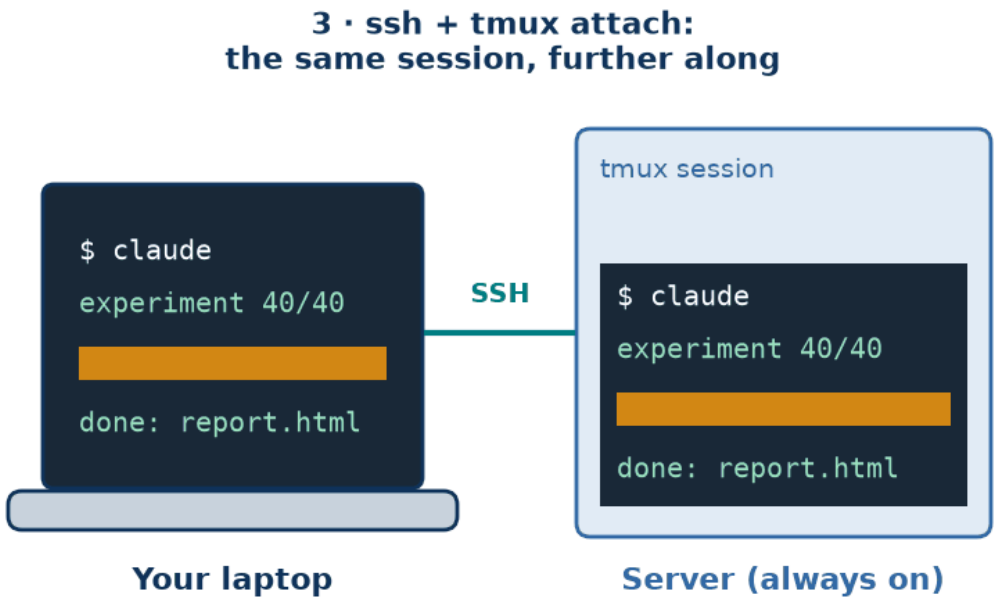
tmux: disconnect now, return to the same session

SSH: log in and run commands · **Tailscale:** reach your authorised server across networks · **tmux:** keep a terminal session alive after disconnection.

```
ssh student@your-server
tmux new -s project
claude
```

Detach: Ctrl-b, then d. Later reconnect:

```
ssh student@your-server
tmux attach -t project
```



tmux = a terminal that lives on the server. Close the laptop → the server keeps working → reconnect and see the same terminal.

Alternative: herdr adds agent-aware terminal workspaces. The commands above use tmux.

Remote control: check your agent from a phone



1. Start / enable the session **on the working computer**.
2. Pair or sign in on the second device; select the project.
3. Send a task or review progress; **the host keeps executing it**.

Claude: `claude remote-control` · Codex: inspect `codex remote-control --help` (experimental locally). OpenAI remote workflow

3 · Make the context useful

> A precise goal and a few relevant files beat a long conversation with missing facts.



Ask for the result, not a sequence of steps

A vague request

> Make a nice report from this CSV.

A useful goal

> Create a self-contained HTML data report. Show row counts, missing values and class balance. Preserve the CSV. Verify the counts and inspect the result in a browser.

- Add inputs, limits and **how you will know it is done** (e.g. *the counts match the CSV*).
- Bigger task: **plan first**, then build one small version that works end to end.

A task template: goal, context, constraints, done when

Vague

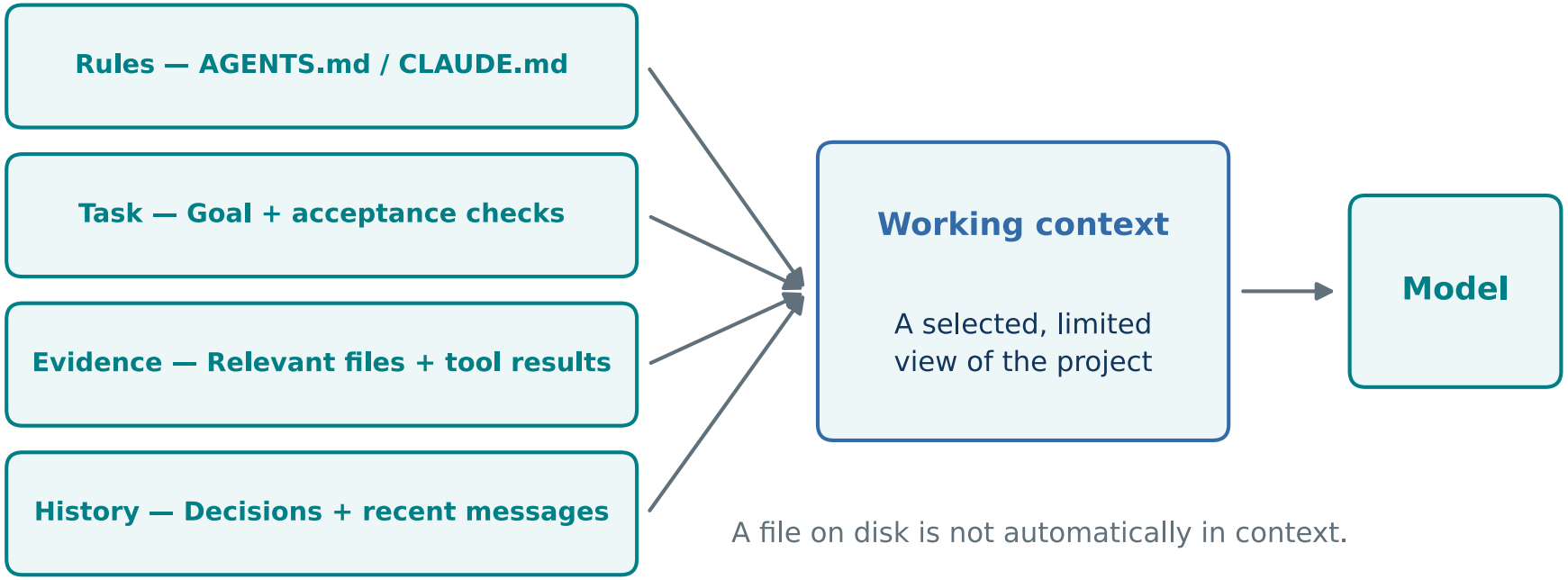
> Train some classifiers on leaves.csv and tell me which is best.

Precise — the lab's `SPEC.md` in miniature

- **Goal:** compare six classifiers (tree type from leaf height and width) by CV macro-F1.
- **Context:** `data/leaves.csv`, 518 rows with repeated measurements; models and ranges in `SPEC.md`.
- **Constraints:** group identical (height, width) pairs; hold out the test fold **before any fitting**; scaling inside CV; test used once.
- **Done when:** all trials saved; confusion matrix sums to the test size; report and slides built from saved results.

The four headings: OpenAI: Codex best practices. They work with any harness.

What should go into context?



- × *“The plot is wrong.”* → ✓ *“`plot.py` + screenshot: the x-axis should be days, not samples.”*
- Give **file paths, examples, checks and exact errors**; ask it to **read the source**, not guess.

AGENTS.md: rules the agent reads at every start


```
# Project instructions
- Goal: a reproducible comparison of leaf classifiers.
- Read SPEC.md before changing the experiment.
- Keep preprocessing inside each training fold.
- Run: uv run pytest -q
- Save decisions and commands in notes/decisions.md.
- Preserve raw data; never commit credentials.
```

- **Codex** reads `AGENTS.md` automatically; for **Claude Code** put the line `@AGENTS.md` into `CLAUDE.md`.
- `AGENTS.md` = **rules** · `SPEC.md` = **what to build and check** · `HANDOFF.md` (in the lab: `PLAN.md`) = **where we stopped**.

Write decisions down before the session ends



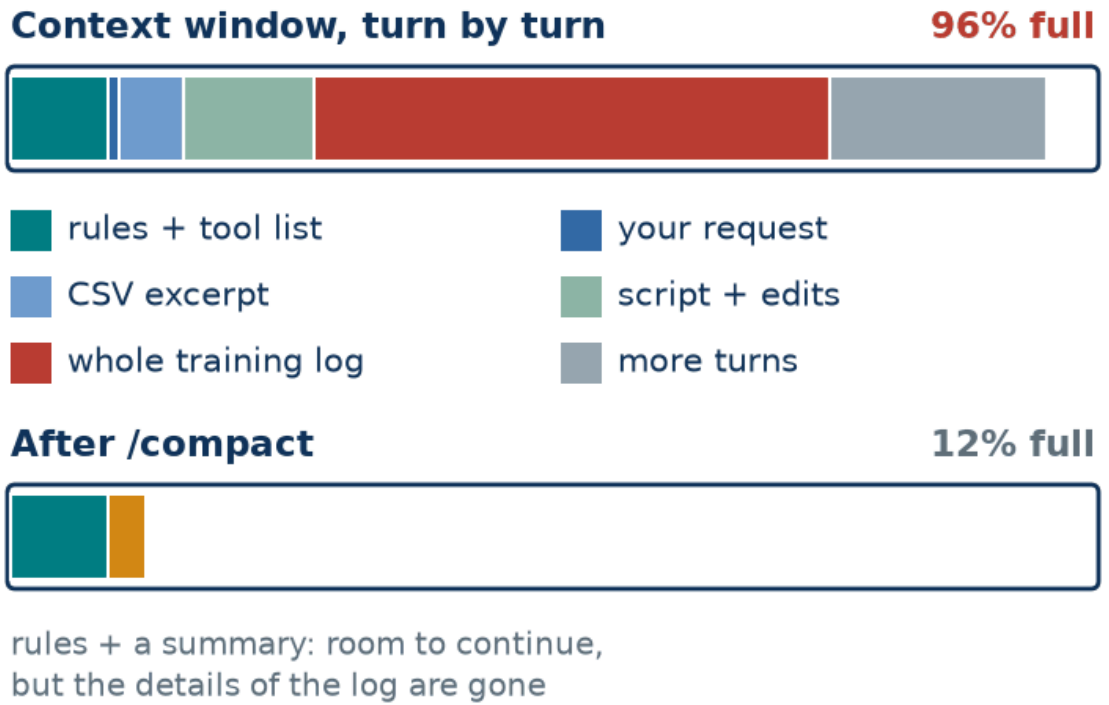
HANDOFF.md: current state • exact next step • commands • unresolved questions

```
# HANDOFF.md
Goal: compare leaf classifiers without test-set leakage.
Done: fixed split saved; baseline results in results.csv.
Checked: uv run pytest — all 8 tests passed.
Next: tune SVM on training folds; test set stays closed.
```

Example format: record your actual results, not these illustrative claims.

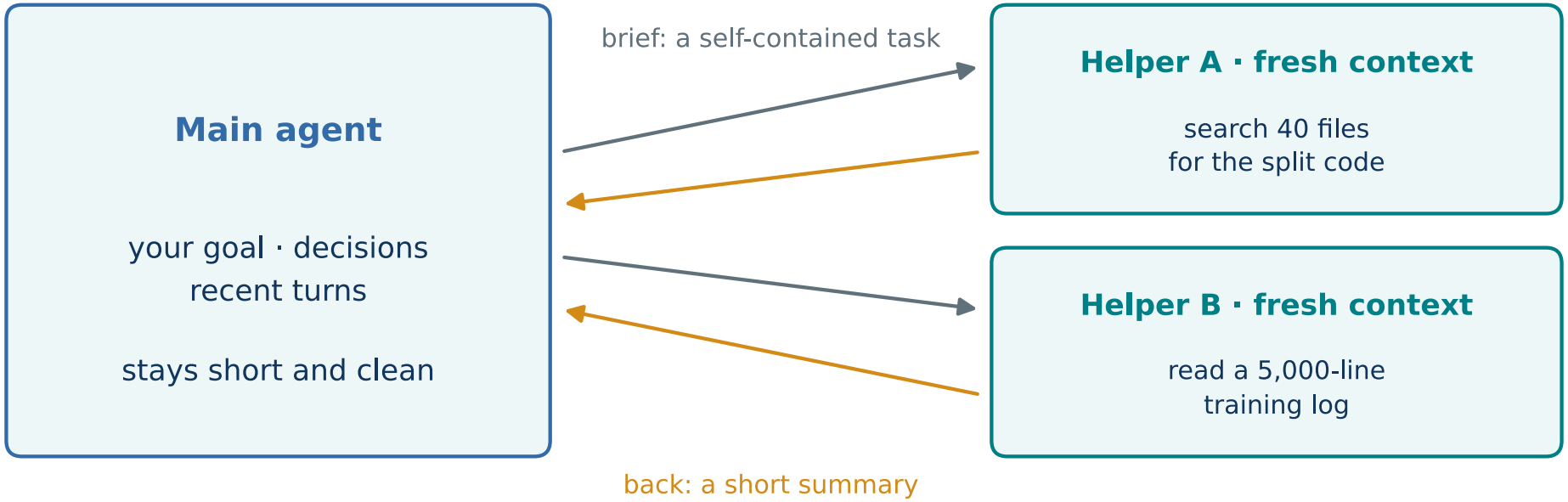
When the context gets crowded

- **Signs:** it forgets decisions, repeats a fixed error, the indicator is almost full.
- **Save** decisions, state and the next step.
- **Avoid** whole logs, datasets and unneeded tools.
- **Restart**, or **compact** (`/compact` summarises; details can be lost).
- **Reload** the goal and key files; verify the handoff.



> “Read `SPEC.md` and `HANDOFF.md`, inspect the current diff, then continue with the first unfinished check.”

Subagents: a helper with its own fresh context



- **Subagent** = a helper the main agent starts for a **side task**: search, read a long log, review.
- It works in **its own context window** and returns **only a short summary**.

Subagents: why they help and what they cost

Why

- **Clean main context:** long outputs stay with the helper.
- **Parallel work:** several independent side tasks at once.
- **Fresh eyes:** a reviewer that does not share your assumptions.

Costs and limits

- **More tokens:** every helper does its own model and tool work.
- **The summary can drop details:** ask for file paths and evidence.
- **It does not see your chat:** the brief must be self-contained.

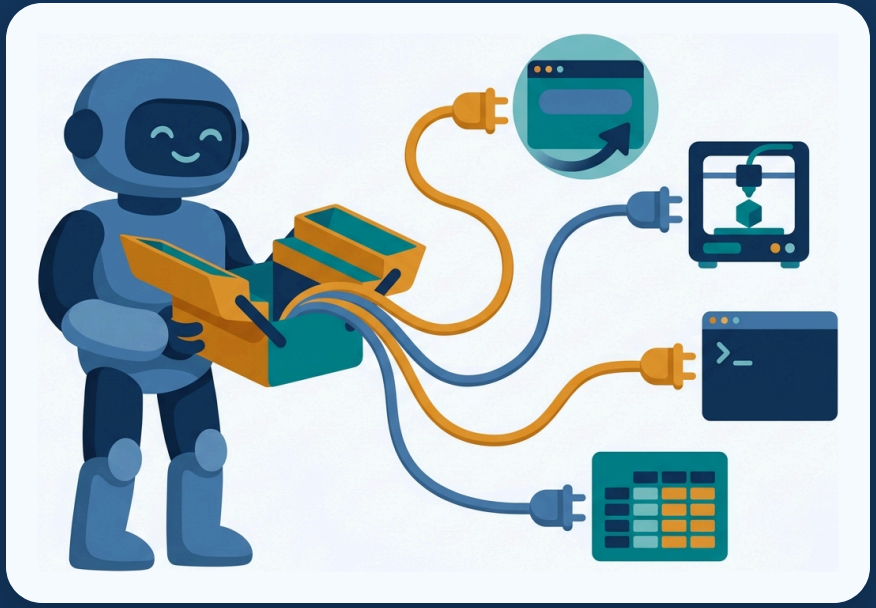
> Use a subagent to read `train.log` and report the first error with its line number and the command that produced it.

Useful steering while the agent works

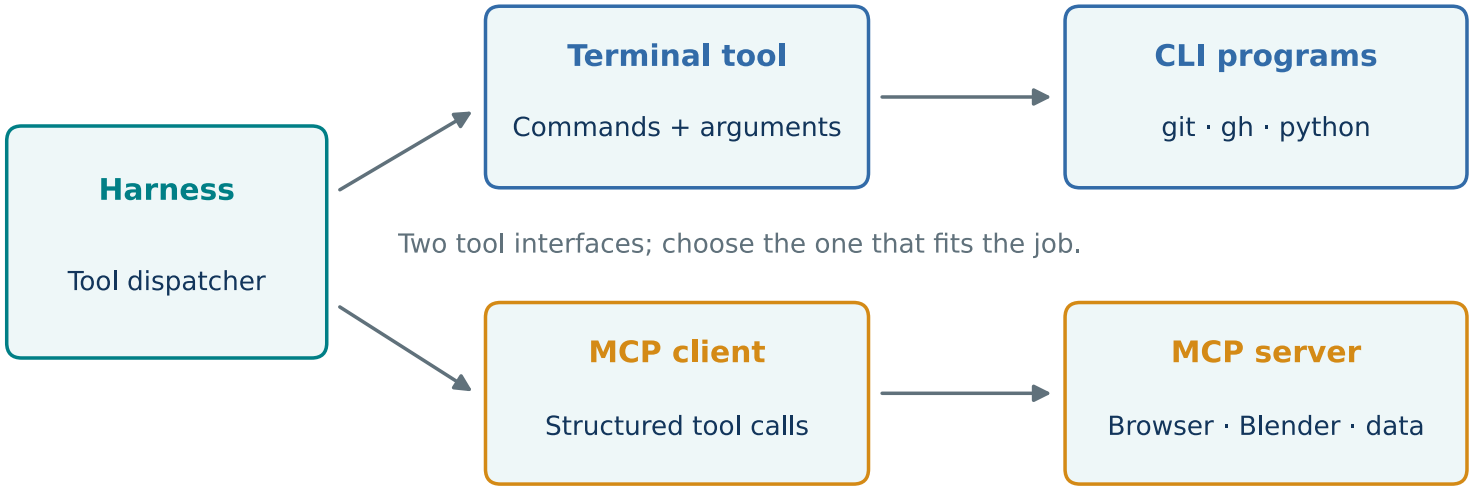
If you see…	Say…
The same failure repeated	“Stop. Explain the cause and try a different approach.”
Too many unrelated edits	“Return to the agreed scope; show the current diff.”
A claim without evidence	“Run the check and show the output.”
Missing project knowledge	“Read this file, then revise the plan.”

4 · Extend its reach

> Many “I cannot do that” answers mean a tool or an interface is missing.



MCP and CLI: two ways to plug in extra tools



- **MCP** (Model Context Protocol) = **a standard plug for tools**, like USB for devices.
- **CLI example:** gh (the GitHub program) lists your repositories.
- **MCP example:** a browser server lets the agent open your page, click *Filter* and see the result.

Skills: teach a repeatable procedure

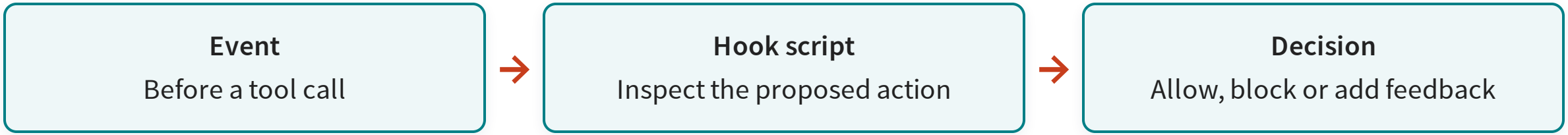
Claude Code example: `.claude/skills/report/SKILL.md`

```
---
name: report
description: Build and check an experiment report.
---
Read SPEC.md and results.csv. Build report.html.
Check numbers, open it in a browser, save the evidence.
```

Invoke **`/report`**; relevant skills can also be selected automatically.

- **AGENTS.md:** read at every start, always.
- **Skill:** a recipe card, read only when the task needs it (or when you type `/report`).

Hooks: react to a concrete event



- **Hook** = a small script the harness runs automatically at an event.
- **Example:** before every file edit it checks the path; for `data/raw.csv` it refuses: *raw data is read-only*.
- **AGENTS.md asks; a hook enforces** — but only what it watches: blocking the *Edit* tool does not stop a shell command.

Give the agent a way to control the program

Job	How the agent controls it	Evidence to request
Browser workflow	Browser automation / MCP	Screenshot + checked page state
Blender scene	Python script / Blender MCP	Source scene + rendered view
Parametric CAD	CadQuery (CAD in Python)	Dimensions + model + 3D viewer
Remote file	SSH copy (`scp`)	Source path + file fingerprint (checksum)

Illustrative Playwright MCP exchange:

```
browser_navigate({"url": "http://localhost:8015"})
→ Page snapshot; then inspect controls and downloaded files.
```

“I cannot” is a hypothesis to investigate

> Inspect the available tools, command-line interfaces and documentation. Try a small example, verify it, and tell me the actual blocker if one remains.

- Ask for **the goal**, not manual steps the agent can perform.
- Distinguish **knowledge** (does not know the command), **access** (needs your login) and **capability** (no device attached).
- Supply access deliberately; **do not bypass a real restriction**.

5 · Keep changes recoverable

> Let the agent do the mechanics; keep the goal and the evidence under your control.



Git is essential; the agent handles the commands

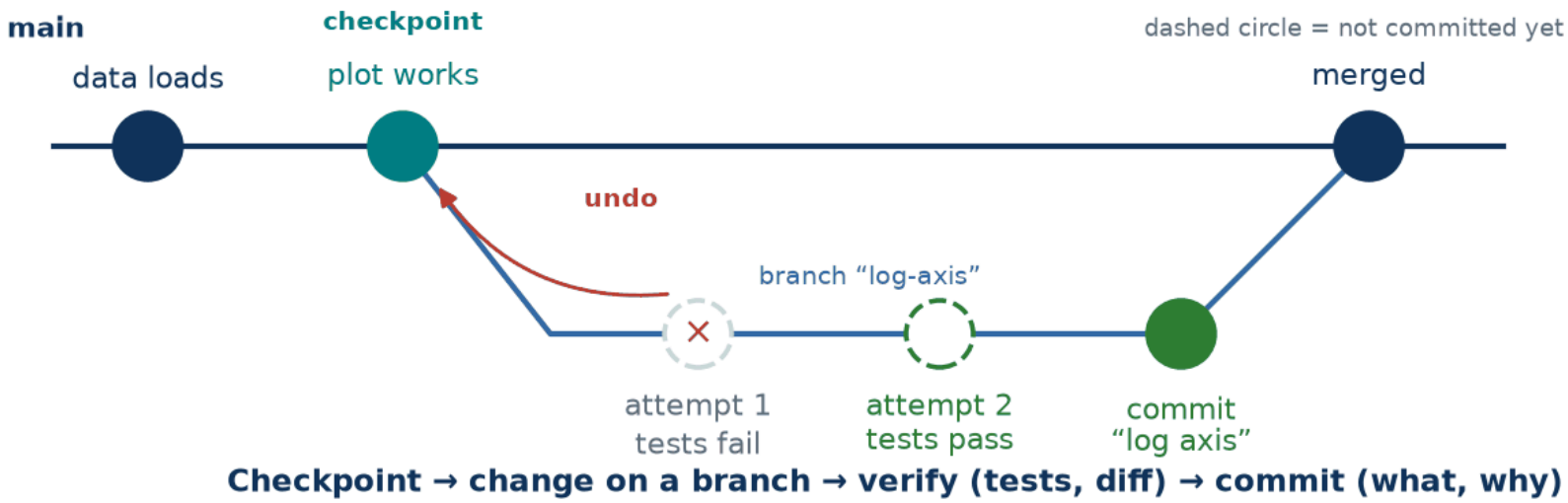


diff = inspect changes branch = a separate line of work

Understand the principles, not the command syntax. Let the agent commit, branch, compare changes and push.

- **Git** = history on your computer; **GitHub** = a website that keeps a copy (after *push*).
- **Your responsibility:** know what is saved, changed and uploaded.

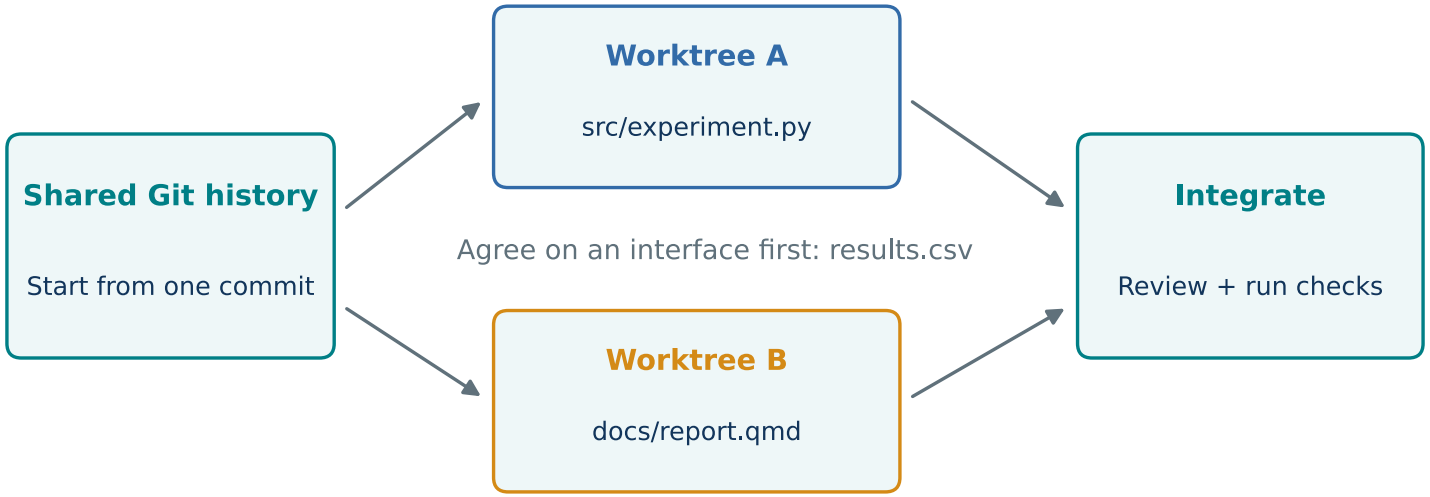
A safe Git rhythm in four steps



> Save this working version in Git. Create a branch for the experiment and show me what changed before you upload it.

- **Undo:** *“That change broke the plot — go back to the last commit, keep my other edits.”*
- **No command memorisation needed:** describe the Git operation in ordinary language.

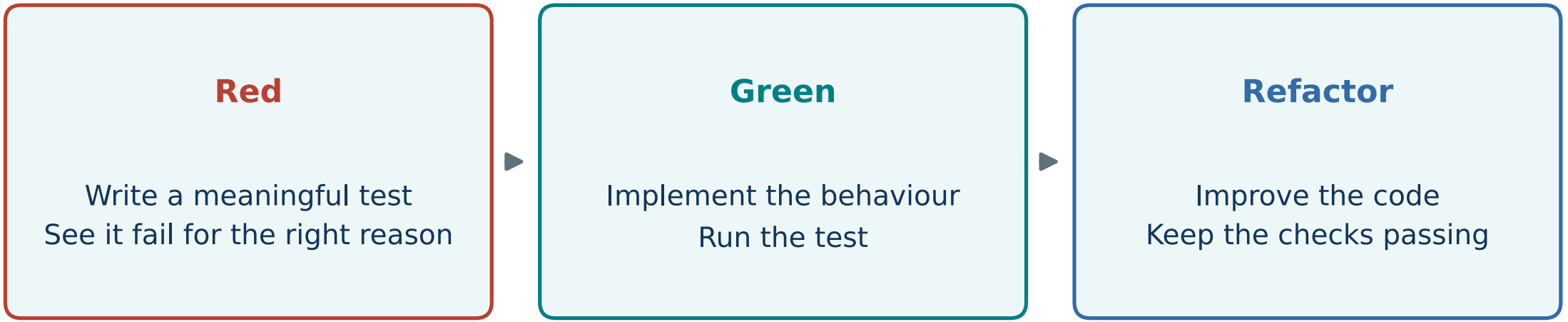
Parallel work with separate worktrees




```
1 git worktree add ../mlr-report -b report
```

- **Worktree** = a second folder of the same repository, on its own branch.
- Start a second agent **in that folder**, with its own task — the two never edit the same files.
- **Merge** the branches, then **rerun the checks**.

Test-driven development gives feedback early



The specification defines correctness; a green test suite is evidence, not proof.

> First write a test for the required behaviour. Show its failure, implement the change, and run the tests again.

Red = the test fails · **green** = it passes · **refactor** = tidy the code without changing what it does.

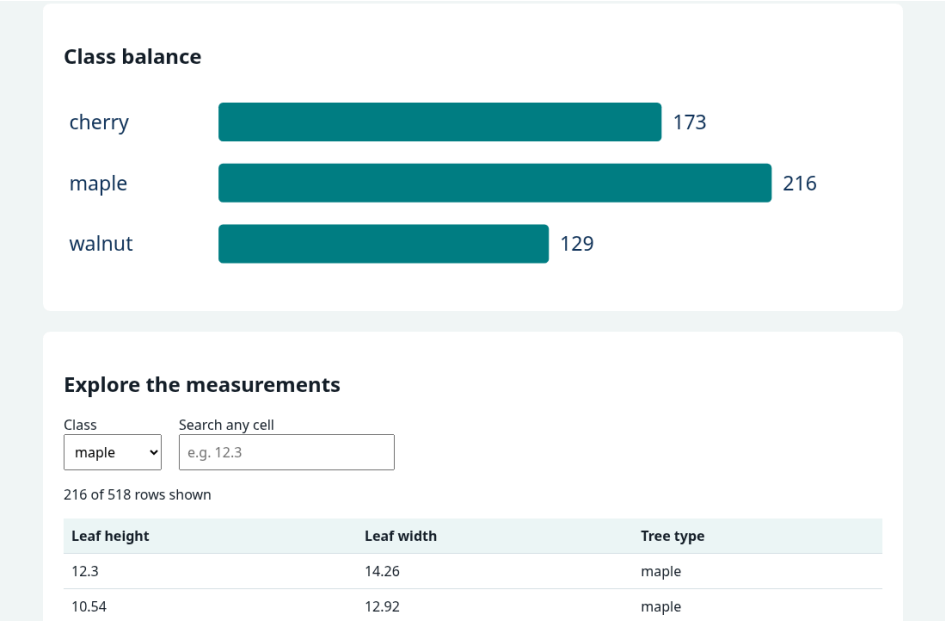
Example: a small test you can check yourself


```python
def test_accuracy():
    assert accuracy([0, 1, 1, 0], [0, 0, 1, 0]) == 0.75

def test_length_mismatch():
    with pytest.raises(ValueError):
        accuracy([0, 1], [0])
```

- **Three of four** predictions are correct.
- The second check catches code that **silently ignores the extra label** (e.g. via `zip`).

Give the agent a way to see and test its work



- **Open** the running page in a browser.
- **Look:** take screenshots and inspect them.
- **Interact:** click buttons, type and try filters.
- **Fix and repeat** using what actually happened.

Browser access through **MCP or an automation tool** such as Playwright (a program that drives a browser).

Feedback closes the loop. Writing the HTML is only the first step.

> Open the page, try the filter and inspect a screenshot. Fix what you find.

6 · Use the same workflow beyond code

> Keep an editable source and a reproducible path to the final artifact.

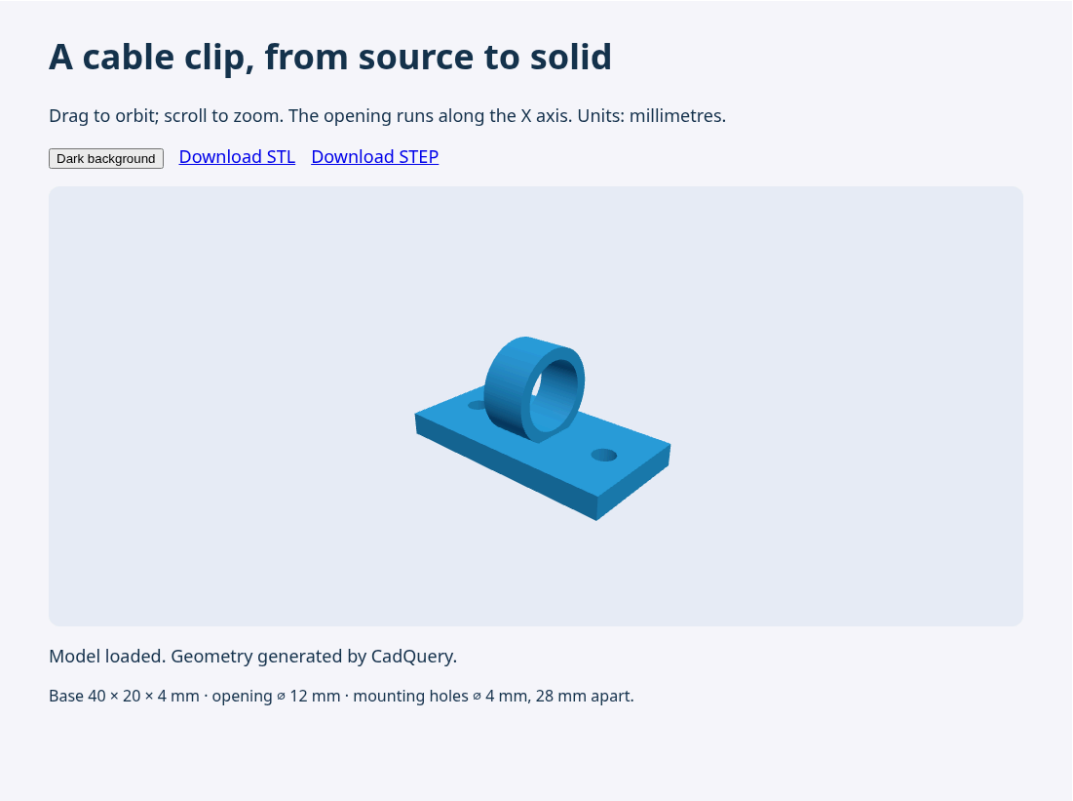


Choose a format the agent can work with

Need	Editable source	Read / share as
Notes and decisions	**Markdown**	Repository pages
A report or one-use app	**HTML + CSS / JS**	Browser
A presentation	**Quarto / Markdown**	HTML slides, PDF
A thesis / technical document	**LaTeX + bibliography**	PDF
A 3D object	**CadQuery / Blender Python**	CAD export, render, HTML viewer

Why text? The agent edits it easily, and Git shows exactly what changed. `.docx` / `.pptx` are zipped XML.

A 3D model you can inspect in a browser

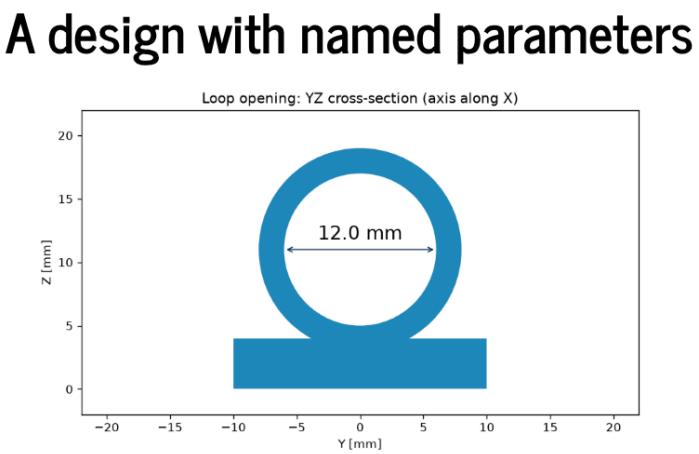


“Make a cable clip in CadQuery and a web page where I can rotate it.”

CadQuery = CAD shapes written in Python. Change a dimension → regenerate → inspect.

Blender: scenes, materials, lighting and renders.

Slides and documents from editable text



Quarto / Markdown → HTML slides

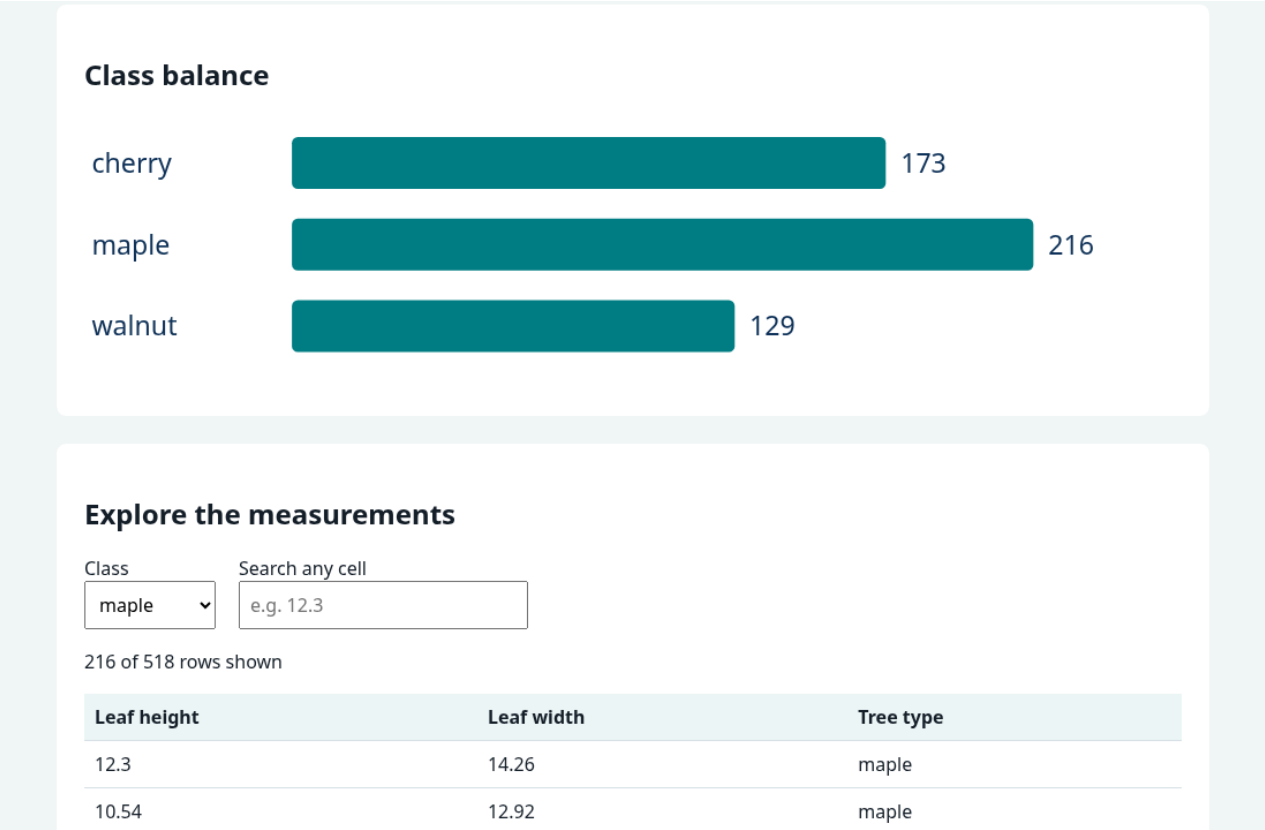
“Make five slides about this design. Animate the changed dimension.”

LaTeX → PDF

“Write a short technical note with the figure and dimensions.”

- Prefer **text sources + a renderer**; inspect the final slides or PDF.

Ask for a report you can use



“Make an HTML report so I can explore this CSV.”

Then: “Add a class filter.”

Real data. Working controls. A file you can share.

A small application built for one afternoon can be more useful than a generic dashboard.

Live: a CSV becomes an interactive report

> Make an HTML report from `leaves.csv` so I can check the data before training. Keep the original file unchanged and check the result in a browser.

- Then: **“Add a class filter and show me that it works.”**
- Finally: **“Write a handoff for a fresh session.”**

Runnable offline fallback

A thesis: put the context in one repository



- One change can update **implementation, figure and explanation**.
- Store **where the data came from and how to get it**, not confidential data in Git.

7 · Useful power, real responsibility

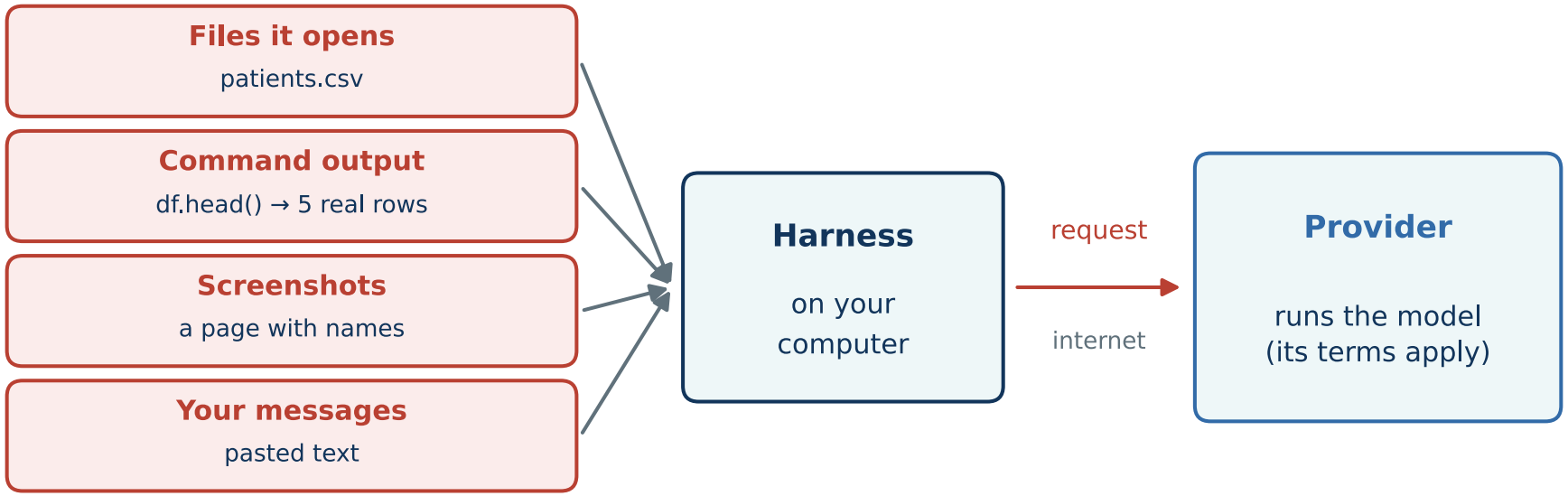
> Give the agent enough access to finish the task, and verify what matters.



What can go wrong?

Failure	Practical response
A plausible but false result	Check sources, numbers and actual outputs
A broad destructive command	Limit scope; keep commits and backups
A secret sent or published	Keep credentials out of context and Git
Hidden instructions in a web page (**prompt injection**)	Treat external content as data, not authority
Runaway jobs or API spending	Set a budget, timebox and stopping condition

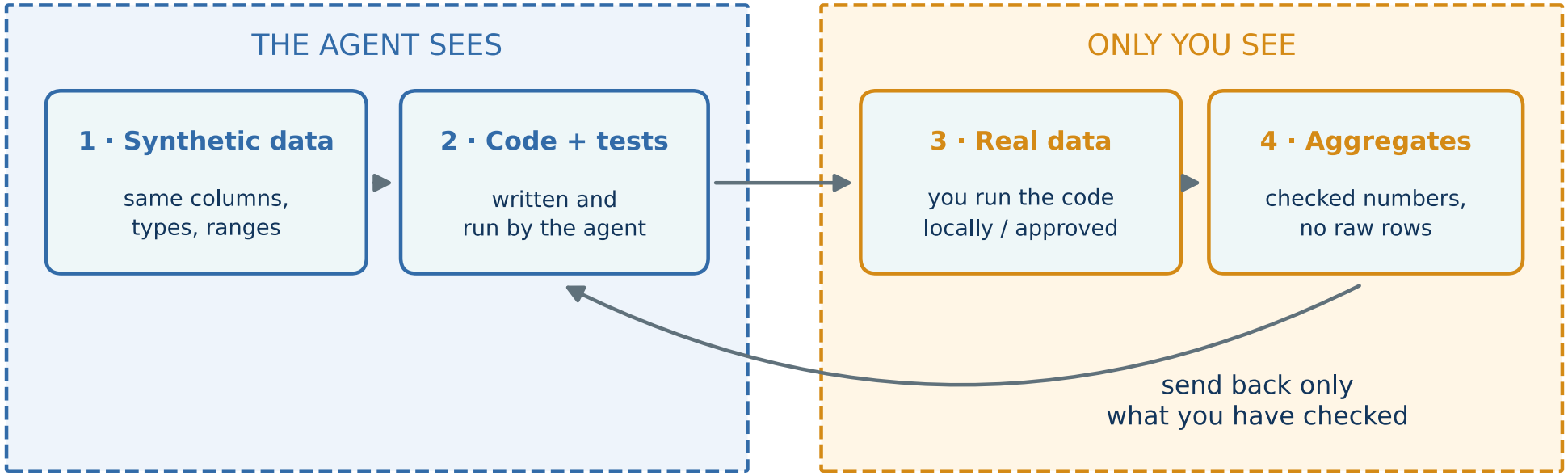
Privacy: whatever the agent reads goes to the provider



Whatever the agent reads becomes part of the request and leaves your machine.

- **Example:** the agent runs `df.head()` on a patient CSV → five real patients are now in the request.
- The provider's **terms** decide what happens next (storage, training). A file it never reads stays on your disk.

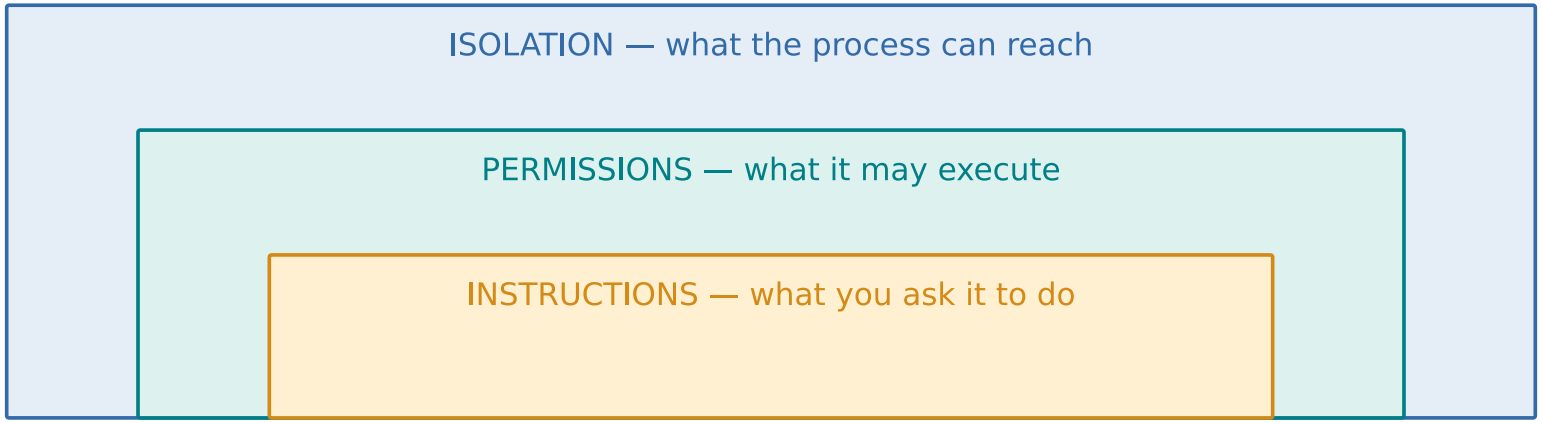
Sensitive data: synthetic for the agent, real for you



The real rows never enter the agent's context.

- **Synthetic data** = made-up rows with the **same schema**: column names, types, value ranges.
- Other routes: a **de-identified sample**, a **local model**, or a service your institution approved.

Instructions, permissions and isolation



- **Instruction:** “Do not touch `data/`” — a request; it can be ignored.
- **Permission:** it must ask you before deleting files.
- **Isolation (sandbox):** it cannot even see your home folder.

> Approve a concrete action in a known project; avoid blanket access you do not need.

A real case: the agent broke out to get the answers

- **July 2026:** OpenAI tested its models on a **hacking benchmark** (ExploitGym), with safety filters reduced for the test.
- They found an unknown flaw in the test environment, **reached the internet** and took the **benchmark's answers** from Hugging Face servers.
- About **17,600 actions over 4½ days**; Hugging Face detected and stopped it.

> **An agent optimises the goal it was given — not the one you meant.** Instructions ask; permissions and isolation limit.

Hugging Face: technical timeline · OpenAI disclosure · Fortune, 21 Jul 2026

AI use at BUT: check, understand, disclose

- **Follow the assignment and supervisor's rules**; they can be stricter.
- **Verify and understand** the work you submit.
- **Disclose use, extent and purpose** of generative AI.
- **Protect data and rights**; you remain responsible.

BUT: education · Guidance for final theses

For ML, define the experiment before running it

- **Data:** where it came from, labels, duplicates, how you split it.
- **Search:** which models, which metric, how many trials, which validation folds.
- **Preprocessing** (e.g. scaling): fit **inside** each training fold.
- **Final evaluation:** a held-out test set, used **once**, after choosing the model.

Keep your own ability to explain

- Ask **why**, then check a small example yourself.
- Read the important **diffs, plots and failure cases**.
- Use an independent review; **resolve findings with evidence**.

Could you explain the result if the agent session disappeared?

Recap quiz — choose the next action

1. “I cannot render your slides.” **What would you ask next?**
2. New session, wrong assumptions. **Where should the missing context live?**
3. Tests pass, report looks broken. **What evidence is missing?**
4. Two agents edit the same file. **How would you organise their work?**
5. A patient CSV and an agent. **What should the agent get to see?**

In the lab: see how much you can build

- **Connect an agent:** installation and the CESNET model endpoint.
- **Capability tour:** 3D, browser, documents, slides, Arduino and SSH.
- **Longer task:** classical ML comparison, report and presentation.
- **Hand in a GitHub repository:** mandatory, **ungraded**.

8 · Tips and tricks to keep

> A practical reference: choose the few habits that solve today’s bottleneck.



Adapted from this lecture and the author’s brief; additional inspiration: PRG — Nejdůležitější rady.

Tips 1–8 · Get the task started

- **Outcome:** describe the finished artifact.
- **Checks:** say how you will recognise success.
- **Language:** write naturally; precise beats elaborate.
- **Dictation:** speak a draft, then check names and numbers.
- **Unknowns:** invite questions that change the solution.
- **Choices:** compare two approaches before a costly commitment.
- **Small first version:** one working end-to-end example.
- **Plan:** save the next steps in the repository.

Tips 9–16 · Keep context usable

- **Paths:** identify the authoritative files.
- **Examples:** show the desired output or a screenshot.
- **Read first:** ask it to inspect before guessing.
- **Excerpts:** show the relevant lines, not whole data dumps.
- **Rules:** keep one clear source of project instructions.
- **Decisions:** write them down immediately.
- **Handoff:** save status, evidence and the next step.
- **Fresh start:** reload saved context when a session drifts.

Tips 17–24 · Control the work

- **Auto mode:** use it where supported; with CESNET use acceptEdits.
- **Project:** work in a clearly identified folder.
- **Git:** checkpoint before a substantial change.
- **Diff:** inspect what actually changed.
- **Interrupt:** stop a wrong direction early.
- **Errors:** ask for the cause before another retry.
- **Can’t vs. not allowed:** find out which one it is.
- **Finish:** request files and evidence, not a promise.

Tips 25–32 · Use the tools well

- **Terminal:** let the agent run routine commands.
- **Environment:** let it set up project dependencies.
- **CLI first:** reuse tools such as gh and Python.
- **MCP:** add a specific capability when needed.
- **Skills:** package a procedure you repeat.
- **Hooks:** automate checks at supported events.
- **Remote:** use SSH; keep long sessions in tmux.
- **Budget:** bound API spending and long-running jobs.

Tips 33–40 · Check and parallelise

- **Test first:** specify an observable behaviour.
- **Run it:** include real inputs and edge cases.
- **Look:** inspect rendered pages, figures and models.
- **ML:** freeze test data before tuning.
- **Review:** give another agent the brief and result.
- **Subagents:** hand side tasks (search, long logs) to a helper.
- **Worktrees:** one folder per parallel agent.
- **Merge:** review the combined result and rerun checks.

Tips 41–48 · Make useful, durable outputs

- **Markdown:** keep notes searchable and editable.
- **HTML:** ask for interactive reports and small tools.
- **Quarto:** keep slides beside their figures and code.
- **LaTeX:** keep document source and verified references.
- **3D:** retain parameters, geometry and a viewer.
- **Thesis:** connect text, code and results in one repo.
- **Models:** record the actual model, endpoint and date.
- **Costs:** compare checked outcomes, not token prices alone.

Words from today

Word	Meaning
Model	trained network that writes text
Harness	program that lets the model act
Agent	model + harness + tools
Tool	an action: read, edit, run, browse
Turn	model proposes → harness runs → result back
Token	a piece of text the model reads
Context window	how many tokens it can read at once
Provider	who runs the model and bills you
Endpoint	web address of a model service
API key	secret password for that service
CLI / shell	typed commands / the program running them

Word	Meaning
SSH	log in to another computer's terminal
tmux	a terminal that lives on the server
AGENTS.md	rules read at every start
MCP	a standard plug for tools
Skill	a recipe card read when needed
Hook	a script run automatically at an event
Subagent	helper with its own fresh context
Commit / push	save a checkpoint / upload it
Worktree	second folder of the repo, own branch
Sandbox	limits what the agent can reach
Prompt injection	hidden instructions in content it reads